ECPFL: Efficient Ciphertext Prefetching for Secure and Low-Communication Federated Learning Using CKKS Homomorphic Encryption

Zhongyang Shi Email: anonymous@example.com

***Abstract*—Cross-device federated learning (FL) enables collaborative model training across millions of mobile and IoT devices without centralized storage of sensitive data. However, frequent model synchronization between clients and servers incurs substantial communication overhead due to bandwidth constraints. While numerous approaches have optimized uplink and downlink communication overhead to accelerate training, most fail to effectively mitigate synchronization overhead while maintaining privacy protection and security against threat models. This paper introduces ECPFL, an innovative and efficient ciphertext-prefetching (ECP) strategy that establishes a secure, low-latency federated learning framework. ECP is the first ciphertext-prefetching protocol that extends plaintext prefetching into the CKKS domain. We also offer a batch view of the total vector, introducing Voting-based Package-level Top-k with Residual Compensation (VPT), which jointly optimizes gradient sparsification and CKKS slot packing. Specifically, clients perform Top-k selection and prefetch at the slot-batch level, and use error-feedback residual buffers to compensate for information loss from dropped coordinates. ECP allows clients in subsequent rounds to pre-download aggregated ciphertext increments, significantly reducing communication volume and accelerating training. Comparative experiments with state-of-the-art methods demonstrate that the proposed solution reduces communication overhead by XXX-fold and shortens training time by XX-fold without compromising accuracy.**

***Index Terms*—Federated learning, homomorphic encryption, CKKS, ciphertext prefetching, gradient sparsification, residual compensation, trusted execution environment**

I. Introduction

FEDERATED Learning (FL) has emerged as a compelling distributed framework, allowing isolated data silos to jointly optimise a global model while keeping raw data strictly localized [1]. Scaling this concept to the network edge, cross-device FL orchestrates training across massive fleets of smartphones, wearables, and IoT sensors. By pushing computation directly to the data source rather than centralising it, this approach has become highly effective in privacy-critical domains, driving recent advancements in medical diagnostics [2], financial risk modeling [3], and intelligent edge systems [4]. Compared to cross-silo FL, cross-device FL faces substantially more stringent constraints. The participating devices may number in the millions, and their network connections are often unstable and bandwidth-limited. Additionally, these devices exhibit significant heterogeneity in computational power and battery capacity. Furthermore, the non-independent and identically distributed (Non-IID) nature of local data results in pronounced statistical heterogeneity [5], [6], [7], [8]. Collectively, these factors present fundamental challenges to the practical deployment of cross-device FL.

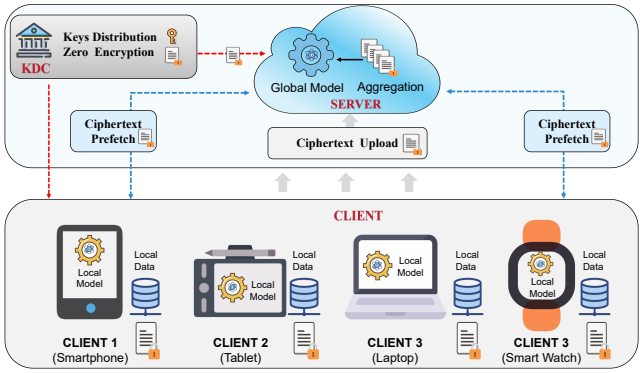


Fig. 1. ECPFL system architecture. The KDC initializes keys and goes offline. Clients upload sparse ciphertext gradients and receive prefetched stacked global ciphertexts from the server. The red line represents a secure channel, and the blue line represents an insecure channel.

First, communication overhead is a major barrier to the large-scale deployment of cross-device FL, which manifests in both upstream (client-to-server) and downstream (server-to-client) directions [9]. For typical deep neural networks, each client needs to upload and download millions to tens of millions of model parameters per round, while uplink bandwidth is often only one-fifth to one-tenth of downlink [10]. To address this, researchers have explored two dimensions: client sampling, in which only a subset of clients participates each round [11], [12], [13], [14], [15], [16] and update compression, which reduces the number of bits per transmission through gradient quantization [17], sparsification [18], low-rank decomposition [19], or sketch-based methods [20]. However, FedFetch [21] reveals that when client sampling with downstream compression, the gains significantly cancel out—unsampled clients suffer from stale local models and require downloading large incremental updates to resynchronize, causing downstream communication to approach full-model transmission after initial rounds. This finding exposes a structural limitation of existing communication-optimisation schemes in heterogeneous, cross-device environments.

Second, privacy leakage remains a significant challenge in cross-device FL. Studies show that exchanging gradients or model updates does not ensure privacy, as adversaries can reconstruct sensitive training data from these updates [22], [23], [24], [25], [26]. To address these risks, three main defense strategies have been developed: differential privacy

(DP) [27], secure multi-party computation (SMPC) [28], and homomorphic encryption (HE) [29]. Among them, HE offers a robust cryptographic basis for secure aggregation by enabling computation on encrypted data. The Cheon-Kim-Kim-Song (CKKS) scheme [30], based on the Ring Learning With Errors (RLWE) assumption [31], is particularly suited for machine learning due to its support for approximate arithmetic and Single Instruction Multiple Data (SIMD) packing. Utilizing ciphertext packing with the Residue Number System and SIMD-style batching, CKKS reduces per-ciphertext overhead and improves throughput compared to slot-wise encryption [32]. Recent systems, including FedPHE [12] and ESAFL [33], use CKKS packing to create efficient secure aggregation protocols, decreasing upstream ciphertext volume through sparsification and polynomial encoding. Multi-key CKKS variants further support aggregation across ciphertexts encrypted with different client keys without decryption [34].

However, existing HE-based FL systems primarily address upstream compression and treat downstream transmission of encrypted global models as a fixed cost. This approach overlooks a critical deployment challenge: all clients are required to synchronously download large, cipher-expanded global models—often several gigabytes under 128-bit CKKS security—over bandwidth-limited networks. Although plaintext FL frameworks FedFetch [21] mitigate this latency by proactively prefetching updates during idle periods, extending this mechanism to the ciphertext domain introduces significant algebraic constraints. Encrypted payloads cannot be partitioned or incrementally accumulated without dimensional expansion or excessive decryption latency. Additionally, pack-level sparsification [35], [36] for upstream optimization causes an algebraic deadlock, as the induced sparsity disrupts key-space alignment required for multi-key homomorphic encryption (MKHE). Current attempts to mask sparsity, such as zero-padding or the use of Trusted Execution Environments (TEEs), either eliminate bandwidth savings or increase the trusted computing base. Therefore, a critical challenge remains: co-designing FL synchronization and ring-level homomorphic operations to simultaneously achieve asynchronous ciphertext prefetching and aligned gradient sparsification without reliance on hardware-based trust.

To bridge this gap, we propose ECPFL, a TEE-free architecture for communication-efficient FL through two co-designed mechanisms: Efficient Ciphertext Prefetching (ECP) and Voting-based Pack-level Top-k (VPTopk). ECP introduces the first asynchronous prefetching in the ciphertext domain by leveraging the additive closure of polynomial rings, allowing clients to prefetch and locally stack aggregated ciphertexts from multiple rounds. Accumulated updates are applied through a single $O(1)$ decryption, eliminating the downstream bottleneck. VPTopk es the algebraic misalignment between gradient sparsification and MKHE through a two-phase consensus: clients first exchange lightweight plaintext bitmaps to negotiate a globally synchronized mask based on local residual magnitudes, followed by symmetric RLWE encryption of selected packets. This approach removes zero-padding overhead, achieves a $1/\zeta$ compression ratio, and uses the multi-key structure as a cryptographic safeguard against server-side isolation attacks. Homomorphic hash signatures further ensure aggregation integrity. ECPFL thus achieves high communication efficiency while maintaining information-theoretic privacy for the aggregation server.

This paper makes the following contributions:

- *Efficient Ciphertext Prefetching (ECP):* We introduce ECP, the first protocol to extend model prefetching to the CKKS ciphertext domain. By aggregating future increments over R rounds with a local ring-stacking accumulator, ECP enables one-shot decryption, reducing amortized downstream communication from $O(d \log q)$ to $O(d \log q/R)$, while maintaining sub-Gaussian noise guarantees.
- *Voting-based Pack-level Top-k with Residual Compensation (VPTopk):* We propose VPTopk, a gradient compression method that structurally aligns Top-k sparsification with CKKS slot packing. By applying ζ-fraction selection and residual feedback at the pack level, VPTopk reduces upstream communication by a factor of $1/\zeta$ and maintains CKKS SIMD aggregation efficiency without incurring repacking overhead.
- *Secure Aggregation Protocol:* We design a five-phase secure aggregation protocol that integrates multi-key encryption, homomorphic hash verification, and anti-tampering consensus voting. This protocol removes mutual trust assumptions among participants and ensures information-theoretic server blindness through offline KDC initialization and forced-alignment verification.
- *Theoretical Proofs and Empirical Validation:* We provide a comprehensive analysis demonstrating semantic security under the decisional RLWE assumption. Moreover, experimental results show that ECPFL achieves up to a $22 imes$ reduction in total communication and a $4.5 imes$ training speedup over state-of-the-art secure FL baselines without sacrificing model accuracy.

Paper Organization. The remainder of this paper is organized as follows. Section II reviews related work in greater depth. Section III introduces the necessary cryptographic and system preliminaries. Section IV presents the detailed design of ECPFL, including the threat model, protocol specifications, and optimization mechanisms. Section V provides formal security analysis and proofs. Section VI evaluates the performance through extensive experiments. Finally, Section VII concludes the paper and discusses future directions.

II. RELATED WORK

This section reviews five lines of research that are most relevant to our work: HE-based federated learning, multi-key homomorphic encryption, gradient compression and sparsification, secure aggregation protocols, and communication optimization in cross-device FL. For each category, we identify the limitations of existing approaches and explain how our proposal addresses them.

Homomorphic Encryption in FL: Homomorphic encryption (HE) provides a mathematically rigorous foundation for privacy-preserving FL. While single-key CKKS schemes,

such as FedSHE [37] and FedPHE [12], support approximate arithmetic and ciphertext packing, they rely on a globally shared key, leaving the system vulnerable to honest-but-curious clients. While multi-Key HE (MKHE) mitigates this by enabling independent key pairs for each client [38], [39], [40]. ESAFL [33] designed a novel one-shot decryption scheme to avoid cross-round attacks by using multiple encryption keys, but it aggregates these keys as the decryption key; however, ESAFL requires all clients to jointly train at the same time. Currently, MASER [41] represents the state-of-the-art MKHE-based FL framework, combining MK-CKKS with consensus-based magnitude pruning. However, MASER relies on magnitude pruning without residual compensation. Unlike MASER, our scheme ensures strict client isolation via MKHE while preserving model accuracy in the presence of data heterogeneity by replacing voting-based pruning with Top-k sparsification and an accumulable residual-error buffer.

Gradient Compression over Encrypted Data: To alleviate the communication bottleneck in FL, various gradient compression techniques have been proposed, including quantization [42], [43], low-rank decomposition [44], [45], and sparsification [46], [47], [48]. Among these, Top-k sparsification with error compensation [49] has proven highly effective. For encrypted FL, DSVFL [50] introduced FTop-k, utilizing an accumulable residual buffer to recover accuracy. However, DSVFL applies sparsification before Paillier encryption, which strictly limits downstream homomorphic operations to simple additions. In contrast, our scheme innovatively integrates Top-k sparsification directly with packed CKKS ciphertexts; moreover, considering a large training set scenario, we further employ the CRTStack method, which dramatically reduces the number of upload ciphertexts. This design not only enables complex homomorphic aggregations on compressed representations but also seamlessly maintains the residual buffer to guarantee convergence.

Secure Aggregation: Traditional secure aggregation protocols heavily rely on pairwise cryptographic masking to protect client updates, such as [28], [51], [52]. While robust against dropouts, these purely software-based protocols incur substantial computation overhead and struggle with malicious server selections [53]. Recently, TEE-based FL frameworks like ExclaveFL [54] and SecFed [34] have emerged to provide hardware-assisted attestation, but they fail to address the inherent computational latency of HE or the staleness in cross-device settings. To bridges this gap, We improve the one-shot decryption [33] scheme as the primary cryptographic privacy mechanism without TEE, achieving true defense-in-depth against both computational adversaries and side-channel attacks.

Communication Optimization via Prefetching: In cross-device FL, the downstream communication (server-to-client) is a critical bottleneck, especially when combining client sampling with compression, which leads to stale local models [55], [56]. FedFetch [21] effectively addresses this by shifting downstream bandwidth usage to a preparation phase via plaintext model prefetching. Our work extends the prefetching paradigm into the ciphertext domain. To the best of our knowledge, this is the first framework to implement ciphertext prefetching in encrypted FL. By allowing clients to proactively fetch compressed ciphertext segments, we successfully overlap the extensive decryption and decompression latency with local training, drastically reducing the end-to-end round time.

Communication Optimization via Client Selection: Client selection is the strategy by which the server chooses $\mathcal{S}^{(t)}$ at each round. It is a critical determinant of both convergence speed and training time in cross-device FL.

1) *Random selection* is a baseline strategy that samples $\mathcal{S}^{(t)}$ uniformly at random from $\mathcal{C}$ [57]. While simple and unbiased, random selection ignores system heterogeneity: clients with low bandwidth or weak compute, whom we also call stragglers, can stall the entire round, and high-utility clients may be undersampled.
2) *Sticky sample* is proposed in GlueFL [16], prioritizes clients that were recently active and have up-to-date model states. The insight is that clients with fresh local models require smaller downstream downloads, reducing per-round synchronization time. This strategy naturally complements prefetching, as pre-sampled clients accumulate state across rounds.
3) *Sketch-based clustering* combined with CKKS was first introduced in FedPHE framework [12]. It has each client compute a low-dimensional sketch of its local model update via locality-sensitive hashing, perturb it for privacy, and send it to the server. The server clusters clients by sketch similarity and selects the fastest client from each cluster.

Actually, ECP inherits the features of the FedFetch architecture and also supports a variety of client sampling strategies, such as *POC* [11], *TiFL* [13], *HACCS* [14], *Oort* [15], and so on. A ciphertext prefetch scheme inherently favors faster clients — with higher bandwidth who can participate more frequently and amortize the prefetch window over more rounds. Therefore, client selection strategies should filter out slow or persistently stale clients.

Summary of Gaps: Existing encrypted FL frameworks exhibit critical limitations: (1) key-sharing vulnerabilities inherent in single-key HE; (2) accuracy degradation and consensus overhead from voting-based pruning in MKHE; (3) decoupled compression and encryption that preclude homomorphic operations directly on sparse representations; and (4) unaddressed downstream communication bottlenecks and model staleness in cross-device settings. We bridge these gaps via a unified architecture synergizing MK-CKKS, residual-compensated Top-k sparsification, verifiable secure aggregation, and pioneering ciphertext prefetching. Table I summarizes the key features of state-of-the-art encrypted FL methods.

III. Preliminaries and motivation

A. Notations

To facilitate a rigorous and unambiguous mathematical formulation of the ECPFL architecture, this section establishes a unified notation system spanning the cryptographic primitives, distributed learning dynamics, and verifiable pipeline mechanics. The notation design adheres strictly to the conventions of

TABLE I
COMPARISON OF PRIOR PRIVACY-PRESERVING FL APPROACHES

Approach	Encryption	Sparsification	Client Selection	Prefetching	Residual Comp.	Voting	Packing
BatchCrypt [58]	Paillier	Quantization	All	✗	✗	✗	Batch
xMK-CKKS [59]	MK-CKKS	✗	All	✗	✗	✗	Slot
FedML-HE [60]	CKKS	Selective enc.	All	✗	✗	✗	Slot
SecFed [34]	BFV	✗	All	✗	✗	✗	✗
ESAFL [33]	MK-CKKS	✗	All	✗	✗	✗	Poly.
FedPHE [12]	CKKS	Obfuscation	Sketch-based	✗	✗	✗	Batch
MASER [61]	MK-CKKS	Magnitude pruning	All	✗	✗	Consensus	Slicing
FAS [62]	CKKS	✗	All	✗	✗	✗	Partial
DSVFL [50]	Paillier	Top-k	All	✗	Error feedback	✗	Share-based
ECPFL (Ours)	**MK-CKKS**	**VPtopk**	**Sticky-based**	✓	**Error feedback**	**Consensus**	**Batch**

modern applied cryptography and systems security literature. Table II summarizes the key notations used throughout this paper.

B. Cross-Device Federated Learning

In cross-device federated learning (FL), each client $i \in [N]$ holds a private local dataset $\mathcal{D}_i$ and trains a model with parameters $w \in \mathbb{R}^d$. At each round t, the server selects a subset $\mathcal{S}_t \subseteq [N]$ of $K = |\mathcal{S}_t|$ clients. Each selected client $i \in \mathcal{S}_t$ performs τ steps of stochastic gradient descent (SGD) on its local data, computes a local gradient g_i^t, and uploads it to the server. The server aggregates the received gradients and updates the global model:

$$w^{t+1} = w^t - \eta \sum_{i \in \mathcal{S}_t} \frac{|\mathcal{D}_i|}{|\mathcal{D}|} g_i^t, \tag{1}$$

where $|\mathcal{D}| = \sum_{i \in [N]} |\mathcal{D}_i|$ and η is the global learning rate. This is the standard FedAvg algorithm [57].

The cross-device setting introduces several practical challenges that distinguish it from cross-silo FL:

- *Limited bandwidth:* Clients (e.g., mobile phones, IoT devices) have restricted upload/download bandwidth BW_i.
- *Client dropout:* Clients may drop out during training due to network instability, battery depletion, or user interruption.
- *System heterogeneity:* Devices vary significantly in compute capability, memory, and network conditions.
- *Non-IID data:* Local datasets $\mathcal{D}_i$ are typically drawn from different distributions, causing slow and unstable convergence.

These challenges motivate the design of efficient, robust, and privacy-preserving protocols tailored to the cross-device setting.

C. CKKS Slot Packing Encryption

The Cheon-Kim-Kim-Song (CKKS) scheme [30] is a leveled homomorphic encryption scheme that supports approximate arithmetic over complex vectors. We adopt CKKS as the underlying encryption scheme for ECPFL because it natively supports SIMD operations on packed vectors and homomorphic addition, which is essential for secure gradient aggregation.

TABLE II
SUMMARY OF KEY NOTATIONS

Notation	Description
N, T, t	Total clients, total rounds, current iteration
J, j	Total batches and batch index
$\mathcal{S}^{(t)}, \mathcal{D}^{(t)}$	Activate and Deactivated client subset in round t
$K_{\text{up}}, K_{\text{down}}$	Upstream and downstream sparsity bounds
$\mathcal{K}_{\text{up}}^{(t)}, \mathcal{K}_{\text{down}}^{(t)}$	Indices for consensus and prefetch batches
$\mathbf{w}_i^{(t)}$	Local model parameters of client i
$\mathbf{g}_{i,j}^{(t)}$	Local gradient of client i at round t for batch j
$v_i^{(t)}, \Pi_{\text{vote}}^{(t)}$	Local voting vector and global consensus proof
R_q, R_p	Ciphertext and plaintext polynomial rings
s_i, s	Client key fragment and global decryption key
$B, a^{(t)}$	Sync seed and round-wise public polynomial
$m_{i,j}^{(t)}, e_{i,j}^{(t)}$	Encoded gradient and Gaussian noise polynomial
$c_{i,j,\text{up}}^{(t)}$	Client-side upstream encrypted gradient
$C_{k,j}^{(t)}$	Compensation ciphertext for inactive clients
$C_{j,\text{down}}^{(t)}$	Global aggregated downstream ciphertext
$C_{\text{pre}}^{(t)}, a_{\text{pre}}^{(t)}$	Prefetch ciphertext and merged public parameter
$\mathbf{\Delta}_{\text{pre}}^{(t)}$	Decrypted multi-round accumulated gradient
$\mathbb{G}, p', \mathbf{g}$	Hash cyclic group, group order and generators
$h_{i,j,\text{up}}^{(t)}$	Homomorphic hash of upstream ciphertext
$H_{j,\text{down}}^{(t)}, H_{\text{pre}}^{(t)}$	Global integrity hash and prefetch hash anchor

1) RLWE Problem: The IND-CPA security of CKKS relies on the decisional Ring Learning With Errors (dRLWE) problem [31]. Operating over the cyclotomic ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$, a ciphertext tuple $(\mathsf{ct}_0, \mathsf{ct}_1) \in \mathcal{R}_q^2$ mathematically encapsulates the secret key $s \leftarrow \chi_s$ and Gaussian noise $e \leftarrow \chi_e$ via the relation $\mathsf{ct}_0 = -\mathsf{ct}_1 \cdot s + e + \mathsf{pt} \pmod q$, where $\mathsf{ct}_1 \leftarrow \mathcal{U}(\mathcal{R}_q)$. This structure strictly dictates our zero-ciphertext compensation design.

2) CKKS Encoding: CKKS leverages an isometric mapping to achieve highly compact spatial encryption. A complex vector $\mathbf{z} \in \mathbb{C}^{n/2}$ is mapped into a single polynomial pt via canonical embedding:

$$\mathsf{pt} = \mathsf{Encode}(\mathbf{z}, \Delta) = \lfloor \Delta \cdot \phi^{-1}(\mathbf{z}) \rceil \in \mathcal{R}_q, \tag{2}$$

where Δ preserves the floating-point precision. Consequently, one CKKS ciphertext can simultaneously pack $n/2$ independent scalar gradients.

3) Additive Homomorphism.: To execute secure aggregations with minimal computational overhead, ECPFL strictly

utilizes the additive homomorphism of CKKS, bypassing expensive homomorphic multiplications:

$$\mathsf{Decode}\big(\mathsf{Decrypt}(\mathsf{ct}_A \oplus \mathsf{ct}_B)\big) \approx \mathbf{z}_A + \mathbf{z}_B. \tag{3}$$

4) Slot Packing: The slot packing capability of CKKS is fundamental to mitigating ciphertext expansion in federated learning. Existing HE-based FL frameworks generally adopt one of two packing paradigms:

Standard SIMD Packing [63], where model gradients are sequentially encoded into the available plaintext slots. While mathematically straightforward, this approach is severely inefficient for sparsified updates: unselected parameters must be padded with encrypted zeros to maintain index alignment, effectively nullifying any bandwidth savings.

Sparse Element Packing [12], which selectively packs only the non-zero gradients into consecutive slots. Although this maximizes individual ciphertext utility, it fundamentally destroys the global spatial alignment across heterogeneous clients, necessitating complex, computationally prohibitive homomorphic repacking or masking operations at the aggregation server.

There are also packing strategies such as *Diagonal Packing* [64], *Multiplexed Packing* [65], *BitStack Packing* [66], and *CRTStack Packing* [67], which are used for efficient encrypted neural network inference.

D. Ciphertext Compression in FL

Homomorphic encryption in FL introduces substantial communication overhead: a single CKKS ciphertext is typically thousands of times larger than the plaintext it encodes. To bridge this gap, three families of compression techniques are commonly employed.

Quantization reduces the precision of gradient values from 32-bit floats to lower-bitwidth representations [42]. Extremal variants such as SignSGD [68] transmit a single bit per gradient coordinate. Quantization can be applied before encryption to reduce the plaintext size or combined with packed encryption to maximize savings.

Sparsification retains only the most "important" gradient coordinates, typically those with the largest magnitude, and zeros out the rest [69]. Let $k \ll d$ be a sparsity budget; top-k sparsification keeps only the k coordinates with the largest $|\mathbf{g}_i[j]|$. Error-feedback mechanisms accumulate the dropped coordinates in a local residual buffer and reinject them in future rounds, ensuring convergence despite aggressive sparsity [69].

Packing. Rather than encrypting each scalar parameter individually, multiple values are packed into a single plaintext polynomial and encrypted jointly [12]. CKKS natively supports $N/2$ slots per ciphertext; CRT-based packing (Sec . III-C4) further extends this by encoding multiple values *within* each slot. The combination of sparsification, CRT packing, and quantization yields ciphertext sizes comparable to, or even smaller than, the original plaintext gradients.

Challenges. While compression reduces per-round communication, it introduces new problems in the encrypted domain: (i) aggressive quantization can destabilize training under non-IID data [3]; (ii) sparsification requires clients to agree on which coordinates to retain, necessitating costly consensus protocols when each client uses a different secret key; and (iii) packing strategies must be carefully matched to the CKKS modulus budget and the multi-key aggregation structure.

E. Ciphertext Pretching in FL

Prefetching has emerged as a promising technique to mitigate communication latency caused by model staleness in federated learning, strategically shifting downstream payload transfers from the critical training path to idle bandwidth periods.

1) Plaintext Prefetching: FedFetch [21] introduces a comprehensive plaintext prefetching framework for cross-device FL. It mitigates staleness by executing a *Prepare phase* (presampling clients) and a *Prefetch phase* (background downloading). Mathematically, if an idle client is scheduled to reparticipate after a horizon of R rounds, it proactively downloads the plaintext global model updates $\mathbf{\Delta}^{(t+r)} = \mathbf{w}^{(t+r+1)} - \mathbf{w}^{(t+r)}$ round-by-round. The client locally accumulates these updates to construct the composite model delta:

$$\mathbf{\Delta}_{\text{pre}}^{(t)} = \sum_{r=0}^{R-1} \mathbf{\Delta}^{(t+r)}. \tag{4}$$

When the client officially joins the training phase, it directly applies $\mathbf{\Delta}_{\text{pre}}^{(t)}$ to synchronize its local model. However, this paradigm inherently assumes that gradients are transmitted in plaintext, rendering it fundamentally incompatible with encrypted FL.

2) Ciphertext Prefetching: Extending the prefetching paradigm to the encrypted domain necessitates a mechanism that mirrors Equation (4) without violating cryptographic boundaries. In our ECPFL architecture, clients also follow the *Prepare* and *Prefetch* phases but operate strictly on MK-CKKS ciphertexts. Instead of aggregating plaintexts, the returning client utilizes its idle bandwidth to download the global aggregated ciphertexts $C_{\text{down}}^{(t+r)}$ generated by the server at each round. Crucially, exploiting the additive homomorphism and the closure property of the cyclotomic polynomial ring R_q, the client performs local accumulation over the prefetched ciphertexts:

$$C_{\text{pre}}^{(t)} = \Big[\sum_{r=0}^{R-1} C_{\text{down}}^{(t+r)}\Big] q \tag{5}$$

This design is algebraically profound: the local homomorphic addition strictly bounds the composite ciphertext $C\text{pre}^{(t)}$ to a single polynomial dimension, achieving zero ciphertext expansion regardless of the prefetch depth R. Consequently, when the client transitions to the training phase, it bypasses the need for R sequential decryptions. Instead, it executes a single $O(1)$ one-shot decryption on $C_{\text{pre}}^{(t)}$ using the global decryption key sequence, seamlessly recovering the precise multi-round plaintext gradient $\mathbf{\Delta}_{\text{pre}}^{(t)}$.

Challenges. Ciphertext prefetching differs fundamentally from its plaintext counterpart in three critical ways:

(1) *No client-side accumulation*: Clients cannot accumulate deltas themselves because they cannot decrypt intermediate ciphertexts. The server performs all accumulation via homomorphic addition.
(2) *Key alignment*: Each round's aggregated ciphertext contains a key-dependent term $a^{(t)} \cdot \sum_{i \in \mathcal{S}^{(t)}} s_i$. Multi-round accumulation produces cross-terms involving different public polynomials $\{a^{(t+r)}\}$ and different participant sets $\{\mathcal{S}^{(t+r)}\}$. Key-compensating cipher injection (KCC) is required to normalize these terms.
(3) *Noise growth*: Each homomorphic addition increases the error polynomial magnitude. Over R rounds, the accumulated noise scales as $R \cdot n_{\max} \cdot B_{\text{err}} \cdot \sqrt{N}$, constraining the maximum prefetch window given the CKKS modulus budget.

F. Motivation

The architectural design of ECPFL draws profound inspiration from recent breakthroughs in communication-efficient FL and applied cryptography, yet fundamentally extends these works to bridge critical deployment gaps. Specifically, the ECP pipeline is inspired by the idle-time synchronization concept of FedFetch [21]. However, we elevate this paradigm from the plaintext environment into the ciphertext domain, resolving the nontrivial algebraic challenges of stacking multi-round encrypted payloads via homomorphic ring addition without triggering dimensional expansion. Furthermore, our cryptographic foundation is built on the elegant multi-key mechanism of ESAFL [33]. While native ESAFL strictly demands full-client participation to maintain key-space alignment, our framework circumvents this limitation. By introducing a globally coordinated consensus and an offline mask compensation mechanism, we successfully decouple the MKHE alignment constraint from dynamic client sampling. Finally, inspired by the secure gradient compression techniques in [12], we advance the methodology by introducing Voting-based Pack-level Top-k. This extension fundamentally upgrades element-wise sparsification to the granularity of CKKS SIMD slots, with strict residual-error feedback, thereby eliminating repacking overhead and seamlessly reconciling aggressive gradient compression with the structural rigidity of homomorphic encryption.

IV. Scheme Design

A. System Model

1) Major Entities: ECPFL operates in a three-entity architecture consisting of a Key Generation Center (KGC), a central aggregation server $\mathcal{S}$, and N distributed clients $\{\mathcal{C}_i\}_{i=1}^{N}$, as illustrated in Fig. 2. The design philosophy departs from conventional federated learning frameworks by introducing an *offline* trusted authority (KGC) and leveraging cryptographic primitives that enable *verifiable blind aggregation* on the server.

- **Key Generation Center (KGC)**: A semi-trusted cryptographic authority responsible solely for distributing global public parameters, static multi-key fragments, and digital certificates during the one-time offline setup phase. It remains entirely offline during the dynamic training process and never participates in active cipher aggregation or zero-filling operations.
- **Central Server** ($\mathcal{S}$): An untrusted or malicious entity responsible for plaintext topology routing, blind aggregation of homomorphic ciphertexts, cryptographic hash verification pooling, and asynchronous ciphertext prefetching scheduling.
- **Heterogeneous Clients** ($\mathcal{U} = \{U_1, U_2, \ldots, U_N\}$): A decentralized network of N clients with data heterogeneity. In any communication round t, the dynamically selected active training cohort is denoted as $\mathcal{S}_{active} \subseteq \mathcal{U}$ (where $|\mathcal{S}_{active}| = K_{act}$). Clients are responsible for local error compensation, sparse bitmap voting, aligned multi-key encryption, homomorphic integrity verification, and one-shot local ciphertext decryption.

2) Protocol Details: The details of ECPFL are shown in Algorithm 1. ECPFL consists of five phases executed in every training round t.

Phase 0: KGC Initialization. The KGC bootstraps the system by sampling independent private key shares $s_i \leftarrow \chi_{\text{key}}$ for each client C_i, deriving public parameters $\mathsf{pp} = (N, q, \tilde{p}, \chi_{\text{key}}, \chi_{\text{err}}, H, B)$, and securely distributing each s_i to its respective client via an authenticated confidential channel. The KGC then permanently goes offline; it neither observes nor participates in any subsequent protocol execution. The public pseudorandom polynomial $a^0 \leftarrow \mathsf{PRPG}(0, B)$ is broadcast to all parties.

Phase 1: Voting Consensus. Each active client locally computes its compensated gradient pack $\tilde{\mathbf{g}}_{i,j}^{(t)} = \mathbf{g}_{i,j}^{(t)} + \mathbf{r}_{i,j}^{(t)}$ (Sec. IV-C), selects the top-k packs via $\textsc{Topk}(\cdot)$, and uploads a binary mask $\mathbf{m}_{i,j}^{(t)} \in \{0,1\}^J$ to $\mathcal{S}$. The server runs majority voting and broadcasts the global consensus mask $\mathcal{K}_{\text{up}}^{(t)}$.

Phase 2: Client Encryption & Homomorphic Hash. Using $\mathcal{K}_{\text{up}}^{(t)}$, clients align their selected gradient packs into fixed-size slots, and encrypt each pack $C_{i,j}^{(t)} \leftarrow \mathsf{CKKS.Enc}(\mathsf{pk}_i, \tilde{\mathbf{g}}_{i,j}^{(t)})$, and compute the homomorphic hash $H_{i,j}^{(t)} \leftarrow \mathsf{HH.Eval}(C_{i,j}^{(t)})$. The tuple $(C_{i,j}^{(t)}, H_{i,j}^{(t)})$ is uploaded to $\mathcal{S}$.

Phase 3: Server Aggregation. $\mathcal{S}$ aggregates the received ciphertexts slot-wise: $C_j^{(t)} = [\sum_{i \in \mathcal{K}_{\text{up}}^{(t)}} C_{i,j}^{(t)} + C_{\text{comp},j}^{(t)}]_q$, where $C_{\text{comp},j}^{(t)}$ is the zero-plaintext compensating ciphertext generated by the KGC for unselected clients (Sec. IV-C). The server also aggregates hashes: $H_j^{(t)} = \prod_i H_{i,j}^{(t)} \mod p'$.

Phase 4: Ciphertext Prefetching & Ring Packing. $\mathcal{S}$ prepares the aggregated ciphertexts and hashes for the current round and a prefetching window of R future rounds. Clients download the prefetch bundle $(C^{(t)}, \ldots, C^{(t+R-1)})$, locally homomorphically accumulate $C_{\text{pre}}^{(t)} = [\sum_{r=0}^{R-1} C^{(t+r)}]_q$, and cache the results for future decryption (Sec. IV-D).

Phase 5: Client Verification & One-shot Decryption. Be-

fore decryption, each client verifies hash integrity $\mathsf{HH.Verify}(C_{\text{pre}}^{(t)}, H_{\text{pre}}^{(t)}) \stackrel{?}{=} 1$. Upon success, it decrypts the accumulated ciphertext in one shot using its private share s_i; otherwise, it aborts and requests a re-transmission.

Algorithm 1: ECPFL Main Procedure

Parameter: Number of rounds T, prefetch horizon R, top-K, vote threshold θ
Input: Local datasets $\{\mathcal{D}_i\}_{i=1}^{N}$, initial model $\mathbf{w}^{(0)}$
Output: Final global model $\mathbf{w}^{(T)}$

▷ Phase 0: KGC Initialization
KGC generates encryption keys s_i, decrytion key S, PRPG seed B, and hash params $(\mathbb{G}, p', \mathcal{H})$
KGC distributes s_i, S and $(\mathbb{G}, p', \mathcal{H})$ to clients U
KGC distributes $(\mathbb{G}, p', \mathcal{H})$ to $\mathcal{S}$
for $t = 1, 2, \ldots, T$ **do**
  ▷ Phase 1: Local Training & Sparse Vote (Client-Side)
  for *each active client* $\mathcal{C}_i$ **do in parallel**
    $\mathbf{g}_i^{(t)} \leftarrow \mathsf{SGD}(\mathcal{D}_i, \mathbf{w}^{(t-1)})$
    $\mathbf{m}_i^{(t)} \leftarrow$ VPTopk-Vote$(\mathbf{g}_i^{(t)}, K)$
    ▷ Alg. 2 Phase A
    Upload mask vote $\mathbf{m}_i^{(t)}$ to server $\mathcal{S}$
  end
  ▷ Phase 2: Global Consensus & Aggregation (Server-Side)
  $\mathcal{S}$ computes $K^{(t)} \leftarrow \{j : \sum_i v_{i,j}^{(t)} \geq \theta |\mathcal{S}^{(t)}|\}$ and broadcasts $K^{(t)}$
  for *each active client* $\mathcal{C}_i$ **do in parallel**
    $\{\mathsf{ct}_{i,j}\}_{j \in \mathcal{K}^{(t)}} \leftarrow$ VPTopk-Encrypt$(\mathcal{K}^{(t)})$
    ▷ Alg. 2 Phase B
    Upload selected sparse ciphertexts to $\mathcal{S}$
  end
  S pads zero-ciphertexts and aggregates: $C_j^{(t)} \leftarrow \sum_{i \in \mathcal{S}^{(t)}} \mathsf{ct}_{i,j}^{(t)} \oplus \sum_{k \in \mathcal{D}^{(t)}} \mathsf{ct}_{k,j}^{(t)}$
  ▷ Phase 3: Ciphertext Prefetching & Synchronization
  if $t \bmod R == 0$ **then**
    Clients and $\mathcal{S}$ execute CipherFetch to sync $\mathbf{w}^{(t)}$ ▷ Alg.3
  end
end
return $\mathbf{w}^{(T)}$

B. Threat Model and Security Goals

ECPFL is designed to withstand a comprehensive threat model encompassing semi-honest clients, a malicious server, and collusion scenarios. Our assumptions closely follow the established multi-key HE-based FL literature [12], [33], [34] while extending them to address the unique challenges introduced by ciphertext prefetching.

Definition 1 (Semi-Honest Client)**.** *Each client C_i honestly follows the protocol specification—computing gradients, generating votes, encrypting updates, and participating in collective decryption—yet attempts to infer private information about other clients' local data from all messages received during protocol execution.*

Malicious Server. The central server S may deviate from the protocol by: *(i)* tampering with aggregation results to bias the global model; *(ii)* selectively dropping ciphertexts from honest clients (a split-attack); *(iii)* substituting prefetched ciphertexts with stale or malformed ones; and *(iv)* inferring private information from traffic patterns or ciphertext metadata. The server, however, remains computationally bounded (PPT adversary) and cannot break the underlying RLWE assumption.

KGC Trust Assumption. The KGC is *semi-trusted* (honest-but-curious): it correctly generates and distributes keying material during Phase 0, but may attempt to retain information after distribution. Critically, the KGC goes *permanently offline* after initialization, eliminating it as a long-term attack vector or single point of compromise. This design choice differs from [12], where the KDC remains online for periodic key refresh.

Collusion Scenarios. We consider two collusion patterns: *(i)* **Client-Client Collusion:** up to $\kappa < n$ clients may pool their private key shares and observed transcripts to attack an honest client's privacy; *(ii)* **Client-Server Collusion:** the server may collude with up to κ clients, combining its aggregation view with their partial key material. Under either pattern, no coalition of fewer than n clients (with or without the server) can decrypt any individual ciphertext or infer an honest client's gradient beyond what is revealed by the final aggregated model update. This $(n-1)$-*resilience* property stems from the fact that the global collective key $\mathbf{S} = \sum_{i=1}^{n} s_i$ is a *virtual* key that never materializes at any single location.

Prefetch-Specific Threats. Ciphertext prefetching introduces two additional threat vectors not present in standard round-by-round aggregation: *(i)* **Side-channel leakage:** prefetched ciphertexts stored locally across R rounds may leak information through timing or access patterns; *(ii)* **Substitution attacks:** a malicious server may replace prefetched ciphertexts with stale ones to delay model convergence. *ECPFL* mitigates *(i)* through round-evolving public polynomials a^t (ensuring fresh randomness per round) and homomorphic hash chaining; threat *(ii)* is addressed by the integrity verification mechanism in Algorithm 3.

This architecture ensures that no single entity can access both the encrypted gradients and the decryption keys, providing strong privacy guarantees against collusion attacks.

C. VPTopk: Voting-Based Sparse Aggregation with Residual Compensation

1) Motivation and Design Rationale: Existing secure aggregation schemes often rely on the unrealistic "full-participation" assumption or impose strong hardware-trust dependencies (e.g., TEEs) to handle client dropouts. ECPFL eliminates both requirements through a pure cryptographic philosophy: clients are categorized into participating and non-participating groups, where the KGC provides offline zero-plaintext compensations for the latter. To maximize the aggregation utility under partial participation, we introduce a forced-aligned sparse encryption mechanism. The detailed client upload procedure is formalized in Algorithm 2, executing in two tightly coupled phases.

2) Error Compensation and Pack-Level Mask Voting: To preserve convergence guarantees under aggressive sparsification, each client $\mathcal{C}_i$ maintains a local residual buffer $\mathbf{r}_i^{(t)} \in \mathbb{R}^d$

(initialized to $\mathbf{0}$). In round t, the client compensates the freshly computed gradient with the previous residual:

$$\tilde{\mathbf{g}}_i^{(t)} = \mathbf{g}_i^{(t)} + \mathbf{r}_i^{(t-1)}. \tag{6}$$

Unlike conventional element-wise sparsification which destroys Single Instruction Multiple Data (SIMD) alignment, VPTopk operates strictly at the pack level. The compensated gradient is partitioned into P packs, denoted as $\{\tilde{\mathbf{g}}_{i,j}^{(t)}\}_{j=1}^{P}$. The client evaluates the significance of each pack by computing its L_2-norm:

$$s_{i,j}^{(t)} = \left\|\tilde{\mathbf{g}}_{i,j}^{(t)}\right\|_2^2, \quad \forall j \in [J]. \tag{7}$$

The client selects the indices of the top-K packs with the largest norms to form the index set $\mathcal{T}_i$. This local selection is encoded into a binary vote bitmap $\mathbf{m}_i^{(t)} \in \{0,1\}^P$, where $m_{i,j}^{(t)} = \mathbf{1}[j \in \mathcal{T}_i]$.

All active clients upload their lightweight plaintext bitmaps to the server. The server executes a global majority vote with threshold $\theta \in (0,1]$ to determine the globally unified pack set $K^{(t)}$:

$$K^{(t)} = \left\{ j \;\middle|\; \sum_{i \in \mathcal{S}^{(t)}} v_{i,j}^{(t)} \geq \theta \cdot |\mathcal{S}^{(t)}| \right\}, \tag{8}$$

This unified set (and its corresponding binary mask vector $\mathbf{M}^{(t)} \in \{0,1\}^J$ with $M_j^{(t)} = \mathbf{1}[j \in K^{(t)}]$) is broadcast back to all clients, enforcing strict spatial alignment across the federation.

3) Forced-Aligned Encryption and Residual Update: Upon receiving $\mathcal{K}^{(t)}$, clients execute the encryption exclusively on the globally selected packs. To radically amortize the ciphertext expansion without violating the approximate nature of CKKS, ECPFL leverages native SIMD batching. The B floating-point scalars within each selected pack $\tilde{\mathbf{g}}_{i,j}^{(t)}$ are tightly mapped into the $N/2$ available plaintext slots via canonical embedding:

$$\mathsf{p}_{i,j}^{(t)} \leftarrow \mathsf{CKKS.Encode}\big(\tilde{\mathbf{g}}_{i,j}^{(t)}, \Delta\big), \tag{9}$$

where Δ is the scaling factor and $\mathsf{p}_{i,j}^{(t)}$ is the encoded plaintext polynomial. This compact spatial encoding perfectly preserves the real-number algebraic structure and avoids the precision corruption inherent to digit-packing schemes.

Subsequently, every pack is encrypted using the client's encryption key $\mathsf{CKKS.Enc}(s_i, \mathbf{v}_{i,j})$:

$$\mathsf{ct}_{i,j}^{(t)} \leftarrow \left[a^{(t)} \cdot s_i + \tilde{p} \cdot e_{i,j}^{(t)} + \mathsf{p}_{i,j}^{(t)}\right]_q, \tag{10}$$

where $a^{(t)}$ is the round-wise public polynomial, $e_{i,j}^{(t)}$ is the Gaussian noise polynomial, and $[\cdot]_q$ denotes reduction over the ciphertext polynomial ring. Equation (10) is the ESAFL encryption primitive; it binds each ciphertext to the client's secret share s_i and the evolving public polynomial $a^{(t)}$, thereby preventing cross-round correlation attacks.

Moreover, to ensure integrity before aggregation, the client generates a homomorphic hash tag for each upstream ciphertext:

$$H_{j,\mathrm{up}}^{(t)} \leftarrow \mathcal{H}\big(c_{i,j}^{(t)}\big), \tag{11}$$

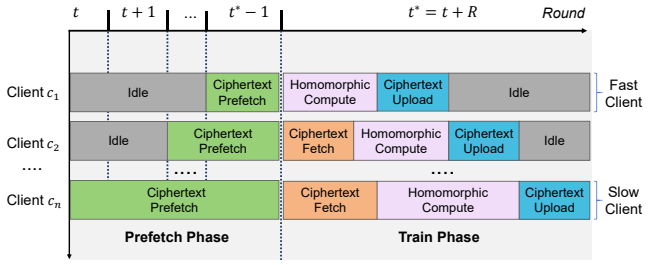


Fig. 2. ECP

where $\mathcal{H}(\cdot)$ denotes the homomorphic hash function. This ensures that the upstream ciphertexts are both perfectly aligned for server-side blind aggregation and mathematically bound to their integrity tags.

Finally, the client updates its local residual buffer. Crucially, the update relies on the *global* consensus mask $\mathbf{M}^{(t)}$ rather than the local vote, ensuring that any pack dropped by the server is accurately retained for future rounds:

$$\mathbf{r}_i^{(t)} = \tilde{\mathbf{g}}_i^{(t)} \odot \big(\mathbf{1} - \mathbf{M}^{(t)}\big). \tag{12}$$

4) KGC Compensation for Dropped Clients: To maintain the required arity for MK-CKKS decryption, the server must pad the aggregation with ciphertexts for any client k. The KGC periodically provisions the server with zero-plaintext compensating ciphertexts evert round:

$$\mathsf{ct}_{k,j}^{(t)} \leftarrow \left[a^{(t)} \cdot s_k + \tilde{p} \cdot e_{k,j}^{(t)} + \mathbf{0}\right]_q. \tag{13}$$

These ciphertexts structurally mimic valid encryptions under the missing secret key s_k, allowing the server to seamlessly complete the homomorphic summation $\sum \mathsf{ct}_{i,j}$ for $j \in \mathcal{K}^{(t)}$.

D. CipherFetch: Effiecient Ciphertext Prefetching Protocol

1) Plaintext Prefetching: FedFetch [21] systematically states the concept of *plaintext prefetching* in FL: the server compresses model updates via Top-k sparsification or quantisation, and clients accumulate these compressed *plaintext* updates locally over a window of R rounds before applying them. The communication volume is substantially reduced because compressed plaintext vectors are far smaller than full ciphertexts.

However, two severe problems exist: first, if we don't apply downstream compression methods, then once most clients' bandwidth is insufficient to prefetch the global update weights, all systems revert to FedAvg; second, plaintext prefetching fundamentally conflicts with secure FL. If the server sends plaintext updates, malicious clients can observe the exact aggregated gradient values of other participants. In our threat model, the server is malicious, and the communication channel is insecure; transmitting plaintext, even compressed, violates the IND-CPA privacy guarantee.

Moreover, for FedFetch, compressed model updates from multiple rounds can be combined before transmission, and the compressed sum may be smaller than the sum of individually compressed updates. Specifically, for masking-based compression, $\delta_{t_1,t_2} = C_{\mathsf{dl}}(\Delta_{t_1}) + C_{\mathsf{dl}}(\Delta_{t_2})$ can be smaller than

Algorithm 2: CRT-VPTopk Client Upload Procedure

Parameter: Number of packs J, top-K, voting threshold θ
Input: Local gradient $\mathbf{g}_i^{(t)}$, previous residual $\mathbf{r}_i^{(t-1)}$, public polynomial $a^{(t)}$
Output: Sparse ciphertexts $\{\mathsf{ct}_{i,j,up}^{(t)}\}_{j\in\mathcal{K}^{(t)}}$, residual $\mathbf{r}_i^{(t)}$

▷ Phase A: Error Compensation & Pack-Level Mask Voting
$\tilde{\mathbf{g}}_i^{(t)} \leftarrow \mathbf{g}_i^{(t)} + \mathbf{r}_i^{(t-1)}$
Partition $\tilde{\mathbf{g}}_i^{(t)}$ into J packs $\{\tilde{\mathbf{g}}_{i,j}^{(t)}\}_{j=1}^{J}$
Compute L_2-norm for each pack $j \in [P]$: $s_{i,j}^{(t)} \leftarrow \|\tilde{\mathbf{g}}_{i,j}^{(t)}\|_2^2$
$\mathcal{T}_i \leftarrow$ top-K indices of $\{s_{i,j}^{(t)}\}_{j=1}^{J}$
Upload vote bitmap $\mathbf{m}_i^{(t)} \in \{0,1\}^P$, where $m_{i,j}^{(t)} = \mathbf{1}[j \in \mathcal{T}_i]$
Receive $\mathcal{K}^{(t)}$ and $\mathbf{M}^{(t)}$ from server
▷ Phase B: Forced-Aligned Encryption & Residual Update
foreach $j \in K^{(t)}$ **do**
  $\mathsf{pt}_{i,j}^{(t)} \leftarrow \mathsf{CKKS.Encode}\big(\tilde{\mathbf{g}}_{i,j}^{(t)}, \Delta\big)$; ▷ SIMD Packing
  $\mathsf{ct}_{i,j,up}^{(t)} \leftarrow \big[a^{(t)} s_i + \tilde{p} e_{i,j}^{(t)} + \mathsf{pt}_{i,j}^{(t)}\big]_q$;
  $H_{j,\mathrm{up}}^{(t)} \leftarrow \mathcal{H}(c_{i,j,\mathrm{up}}^{(t)})$
end
$\mathbf{r}_i^{(t)} \leftarrow \tilde{\mathbf{g}}_i^{(t)} \odot (\mathbf{1} - \mathbf{M}^{(t)})$ ▷ Update dropped packs
; **return** $\{c_{i,j,\mathrm{up}}^{(t)}, H_{j,\mathrm{up}}^{(t)}\}_{j\in K^{(t)}}, \mathbf{r}_i^{(t)}$

transmitting δ_{t_1,t_1} and δ_{t_2,t_2} separately because overlapping zero-masks collapse. However, under CKKS homomorphic encryption, the ciphertext size is *fixed* by the ring dimension N and modulus q, independent of the plaintext content. Homomorphic addition of two ciphertexts produces a ciphertext of the *same size* as either input, which is $2{\cdot}N{\cdot}\lceil\log_2 q\rceil$bits. Consequently, ciphertext prefetching cannot reduce communication volume through content-agnostic compression. Instead, its efficiency gain derives from *amortizing the fixed per-ciphertext overhead over R rounds of gradient accumulation*: rather than downloading one aggregated ciphertext per round, each client downloads one prefetched ciphertext every R rounds, reducing the *effective* per-round downstream bandwidth by a factor of R.

To solve these problems, we first design CRT-VPTopk as shown in Alg 2. It aims to reduce the upload size in order to reduce the aggregation size. Then preserves the communication savings of prefetching without sacrificing security. However, one-shot decryption relies on all clients jointly training to aggregate. The question is: we employ package-level top-k even when all clients are online, but their upload batches do not share the same index, which also causes misaligned ciphertexts, making it impossible to apply one-shot decryption. Voting strategy is a good way to solve the problem; research [70] applied voting pruning in the same way. Therefore, with upstream support handling the complex ciphertext operations, ECP can mainly focus on the *Prepare phase* and *Prefetch phase* without worrying about a few clients that may not prefetch the updated global weights.

2) Ciphertext Prefetching: To enable seamless homomorphic addition across consecutive rounds, the server must first ensure that each round's aggregated ciphertext algebraically aligns with the exact same global key structure $S = \sum_{i=1}^{N} s_i$, despite dynamic client dropouts.

Let $\mathcal{S}^{(t)}$ denote the active clients and $\mathcal{D}^{(t)}$ denote the deactivated clients in round t. For any globally selected pack $j \in \mathcal{K}^{(t)}$, the server securely injects the KGC-provisioned zero-plaintext compensations $\mathsf{ct}_{\mathrm{comp},k}^{(t)}$ alongside the active ciphertexts $\mathsf{ct}_{i,j}^{(t)}$. The spatially aligned ciphertext for round t is formulated as:

$$\begin{aligned} C_j^{(t)} &= \sum_{i\in\mathcal{S}^{(t)}} \mathsf{ct}_{i,j}^{(t)} \oplus \sum_{k\in\mathcal{D}^{(t)}} \mathsf{ct}_{k,j}^{(t)} \\ &= \Bigg[a^{(t)} \underbrace{\Big(\sum_{i\in\mathcal{S}^{(t)}} s_i^{(t)} + \sum_{k\in\mathcal{D}^{(t)}} s_k^{(t)}\Big)}_{S} + \tilde{p} E_j^{(t)} + \underbrace{\sum_{i\in\mathcal{S}^{(t)}} m_{i,j}^{(t)}}_{\text{Active Gradients}}\Bigg]_q \\ &= \Big[a^{(t)} S + \tilde{p} E_j^{(t)} + \sum_{i\in\mathcal{S}^{(t)}} m_{i,j}^{(t)}\Big]_q. \end{aligned} \tag{14}$$

where $E_j^{(t)} = \sum_{i\in\mathcal{S}^{(t)}} e_{i,j}^{(t)} + \sum_{k\in\mathcal{D}^{(t)}} e_{k,j}^{(t)}$ is the aggregate bounded noise polynomial.

Subsequently, instead of downloading updates round-by-round, the client prefetches a bundle of R consecutive rounds and performs *local homomorphic accumulation*. Due to the size-invariant property of CKKS, stacking R rounds compresses the temporal dimension into a single composite ciphertext $C_{\mathrm{Pref},j}^{(t)}$:

$$C_{\mathrm{pre},j}^{(t)} = \sum_{r=0}^{R-1} C_j^{(t+r)} = \Big[\Delta_a^{(t)} S + \sum_{r=0}^{R-1} \sum_{i\in\mathcal{S}^{(t+r)}} m_{i,j}^{(t+r)} + E_{\mathrm{pre}}^{(t)}\Big]_q, \tag{15}$$

where $\Delta_a^{(t)} = \sum_{r=0}^{R-1} a^{(t+r)}$ is the accumulated public-polynomial drift and $E_{\mathrm{pre}}^{(t)} = \sum_{r=0}^{R-1} E^{(t+r)}$ is the composite noise. The prefetch ciphertext $C_{\mathrm{pre},j}^{(t)}$ occupies the same asymptotic space as a single-round ciphertext, regardless of the prefetch horizon R.

Simultaneously, the homomorphic hashes are multiplicatively fused to form the verification tag:

$$H_{\mathrm{pre},j}^{(t)} = \mathcal{H}\Big(C_{\mathrm{pre},j}^{(t)} \parallel \Delta_a^{(t)} \parallel t \parallel t+R\Big) \in \mathbb{G}, \tag{16}$$

where $\mathcal{H}$ is the hash function over the cyclic group $\mathbb{G}$ of prime order p'.

3) Client Verification and One-Shot Decryption: Prior to decryption, the client rigorously authenticates the accumulated bundle against the server's signature $\sigma_{\mathrm{Pref}}^{(t)}$. If the integrity check $\mathsf{Verify}(H_{\mathrm{Pref},j}^{(t)}, \sigma_{\mathrm{Pref}}^{(t)}) \neq 1$ fails (indicating server-side tampering, omission, or malicious noise injection), the client deterministically aborts.

Upon successful verification, the client executes the one-shot decryption. Unlike conventional distributed decryption protocols that demand interactive multi-round share exchanges, ECPFL mathematically factors out the global key S in exactly $O(1)$ time relative to the prefetch horizon R. By supplying the global key S, the composite ciphertext effortlessly decrypts $\mathsf{CKKS.Dec}(S, C_{\mathrm{Pref}}^{(t)})$:

$$\begin{aligned} \mathbf{\Delta}_{\mathrm{pre},j}^{(t)} &= \Big[\big[C_{\mathrm{pre},j}^{(t)} - \Delta_a^{(t)} S\big]_q\Big]_p \\ &= \sum_{r=0}^{R-1} \sum_{i\in\mathcal{S}^{(t+r)}} m_{i,j}^{(t+r)} + \tilde{E}_{\mathrm{pre}}^{(t)}, \end{aligned} \tag{17}$$

Algorithm 3: ECP Download Procedure

Parameter: Prefetch horizon R, modulus q, hash prime p'
Input: Server ciphertext stream $\{C^{(t+r)}\}_{r=0}^{R-1}$, hash stream $\{H^{(t+r)}\}_{r=0}^{R-1}$
Output: Updated local model $\mathbf{w}_i^{(t)}$

▷ Server-Side: Homomorphic Accumulation
Init $C_{\text{Pref}}^{(t)} \leftarrow C^{(t)}$ and $H_{\text{Pref}}^{(t)} \leftarrow H^{(t)}$
for $r = 1, \ldots, R-1$ **do**
  $C_{\text{Pref}}^{(t)} \leftarrow C_{\text{Pref}}^{(t)} \oplus C^{(t+r)}$
  $H_{\text{Pref}}^{(t)} \leftarrow (H_{\text{Pref}}^{(t)} \cdot H^{(t+r)}) \bmod p'$
end
Generate validation signature $\sigma_{\text{Pref}}^{(t)} \leftarrow \mathsf{Sign}(H_{\text{Pref}}^{(t)})$
Broadcast bundle $\langle C_{\text{Pref}}^{(t)}, H_{\text{Pref}}^{(t)}, \sigma_{\text{Pref}}^{(t)} \rangle$ to online clients
▷ Client-Side: Verification & Decryption
foreach *online client* $\mathcal{C}_i \in \mathcal{S}^{(t)}$ **do**
  if $\mathsf{Verify}(H_{\text{Pref}}^{(t)}, \sigma_{\text{Pref}}^{(t)}) \neq 1$ **then abort**
  $\mathbf{h}_{\text{chk}} \leftarrow \mathcal{H}(C_{\text{Pref}}^{(t)})$
  if $\mathbf{h}_{chk} \not\equiv H_{\text{Pref}}^{(t)} \pmod{p'}$ **then abort**
  $\mathbf{\Delta}_{\text{pre}}^{(i)} \leftarrow \mathsf{CKKS.Dec}(S, C_{\text{Pref}}^{(t)})$
  $\mathbf{m}_{\text{sum}} \leftarrow \mathsf{CRTRecon}(\mathbf{\Delta}_{\text{pre}}^{(i)})$
end
$\mathbf{w}_i^{(t)} \leftarrow \mathbf{w}_i^{(t_{\text{sync}})} - \eta \cdot \hat{\mathbf{m}}_{\text{sum}}$
$C_{\text{pre}}^{(i)} \leftarrow \emptyset$; $t_{\text{sync}} \leftarrow t$
return $\mathbf{w}_i^{(t)}$

where $\tilde{E}_{\text{pre}}^{(t)}$ is the residual decryption noise. The drift cancellation $-\Delta_a^{(t)} \cdot S$ exactly neutralises the key-mask components accumulated over R rounds, leaving only the summation of active gradients.

Ultimately, the client extracts the exact summation of R rounds of active gradients and applies it to the local model: $\mathbf{w}^{(t+R)} \leftarrow \mathbf{w}^{(t)} - \eta \cdot \mathbf{m}_{\text{sum}}$. This mechanism ensures that neither the server nor the client can ever expose or reverse-engineer the individual client updates or intermediate round states, preserving IND-CPA security unconditionally throughout the prefetch horizon.

V. Security Analysis

ECPFL guarantees end-to-end privacy and verifiable integrity under a formal cryptographic threat model. We provide detailed security analysis across five critical adversarial dimensions:

1) Resilience Against Semi-Honest Clients:

Definition 2 (ECPFL Encryption Scheme)**.** *The ECPFL encryption scheme for client C_i, batch j, and round t consists of the following algorithms:*

- $\mathsf{KeyGen}(1^\lambda)$*: Sample* $s_i \leftarrow \chi_{key} = \{-1, 0, 1\}^N$.
- $\mathsf{Encrypt}(m_{i,j}^t, s_i, a^t)$*: Sample* $e_0, e_1 \leftarrow \chi_{err}$ *and output*

$$c_{i,j,0}^t = [a^t \cdot s_i + \tilde{p} \cdot e_0 + m_{i,j}^t]_q, \tag{18}$$

$$c_{i,j,1}^t = [-a^t + \tilde{p} \cdot e_1]_q. \tag{19}$$

- $\mathsf{Decrypt}(c, \{s_i\}_{i \in [n]})$*: Collective decryption via partial decryption shares (Section* **??***).*

Theorem 1 (IND-CPA Security)**.** *Under the $\mathrm{RLWE}_{d,q,\chi_e}$ assumption, the client encryption scheme in ECPFL is* IND-CPA secure*. Formally, for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$,*

$$\left|\Pr\left[Exp_{\mathcal{A}}^{ind\text{-}cpa}(\lambda) = 1\right] - \tfrac{1}{2}\right| \leq negl(\lambda). \tag{20}$$

Proof. We proceed via a sequence of hybrid arguments that gradually transform the challenge ciphertext into a uniformly random element.

Game$_0$: The real IND-CPA game. The challenger generates keys $(s_1, \ldots, s_n)$, encrypts challenge message m_b as

$$c_b^* = \left[a^{(t)} \cdot s_i + \tilde{p} \cdot e + m_b\right]_q, \tag{21}$$

and sends c_b^* to $\mathcal{A}$.

Game$_1$: Replace the public polynomial $a^{(t)}$ with a uniformly random $a' \leftarrow \mathcal{R}_q$. By the RLWE assumption, $(a^{(t)}, a^{(t)} \cdot s_i + \tilde{p} \cdot e)$ is indistinguishable from (a', u') where $u' \leftarrow \mathcal{R}_q$.

Game$_2$: Replace the challenge ciphertext with a uniformly random $c^* \leftarrow \mathcal{R}_q$. Since $a' \cdot s_i + \tilde{p} \cdot e$ is pseudorandom under RLWE, adding m_b preserves uniformity:

$$c^* = u' + m_b \approx_c \text{Uniform}(\mathcal{R}_q). \tag{22}$$

In Game_2, the ciphertext carries no information about b, so $\Pr[\text{Game}_2 = 1] = 1/2$. By the hybrid argument,

$$\text{Adv}^{\text{ind-cpa}}[\mathcal{A}] = \left|\Pr[\text{Game}_0 = 1] - \tfrac{1}{2}\right| \leq \text{negl}(\lambda). \tag{23}$$

The secret sharing property ensures that any subset of $n-1$ key shares $(s_1, \ldots, s_{n-1})$ reveals *no information* about the global secret $s = \sum_i s_i$. The reconstruction requires all n shares via $s = s_1 + s_2 + \cdots + s_n$. Thus, no coalition of $n-1$ semi-honest clients can decrypt any target ciphertext. □

2) Resilience Against Malicious Server Attacks:

Theorem 2 (Malicious Server Resistance)**.** *If the server drops, replaces, or tampers with any client's ciphertext $c_{i,j}^{(t)}$ before aggregation, the homomorphic hash verification will detect the modification with overwhelming probability $1 - 1/p'$.*

Proof. Each client computes a homomorphic hash over its ciphertext before submission:

$$h_{i,j}^{(t)} = \mathcal{H}(c_{i,j}^{(t)}) = \prod_{k=0}^{d-1} g_k^{c[k]} \bmod p', \tag{24}$$

where $g_0, \ldots, g_{d-1}$ are generators of a multiplicative subgroup of $\mathbb{Z}_{p'}^*$. The server aggregates hashes along with ciphertexts:

$$H_{\text{pre}}^{(t)} = \prod_{i \in \mathcal{S}} \prod_{r=0}^{R-1} h_{i,j}^{(t+r)} \mod p'. \tag{25}$$

By the multiplicative homomorphism of $\mathcal{H}$, the reconstructed pre-fetched ciphertext $C_{\text{pre}}^{(t)} = \sum_{r=0}^{R-1} C^{(t+r)}$ must satisfy

$$\mathcal{H}(C_{\text{pre}}^{(t)}) \stackrel{?}{=} H_{\text{pre}}^{(t)} \mod p'. \tag{26}$$

If the server arbitrarily replaces $c_{i,j}^{(t)}$ with a poisoned ciphertext $\hat{c}$, the discrete logarithm hardness ensures that $\mathcal{H}(\hat{c}) \neq h_{i,j}^{(t)}$ except with a collision probability of $1/p'$, which is strictly negligible for a cryptographically large prime p'. Similarly, if the server selectively drops $c_{i,j}^{(t)}$ to execute a differential

isolation attack [71], the left-hand side of Eq. (26) will lack the corresponding homomorphic contribution, deterministically causing the verification to abort.

Furthermore, a signed hash chain prevents temporal ciphertext tampering attacks: each client signs $h_{i,j}^{(t)} \| h_{i,j}^{(t-1)} \| \text{epoch}_t$ with its private signing key. The server cannot forge signatures or present inconsistent views to different clients without breaking the signature scheme's EUF-CMA security. □

3) Resilience Against Collusion:

Theorem 3 (Collusion Resistance)**.** *The ECPFL protocol satisfies $(n-1)$-collusion resistance: any coalition of $n-1$ clients cannot decrypt a target ciphertext. Even if a client colludes with the server, individual client plaintexts remain hidden unless* all *n secret shares are compromised.*

Proof. Client-only collusion. Suppose clients $\{1, \ldots, n-1\}$ collude, pooling their secret shares $\{s_1, \ldots, s_{n-1}\}$ and observing all ciphertexts. From Eq. (14), decrypting requires the global secret $s = \sum_{i=1}^{n} s_i$. With only $n-1$ shares, the missing share s_n is information-theoretically hidden: for every possible $s_n \in \mathcal{R}_q$, there exists a consistent s_n yielding a valid decryption. Thus, the conditional entropy $H(s \mid s_1, \ldots, s_{n-1}) = H(s_n) = \log_2 |\mathcal{R}_q|$, and the coalition gains no advantage.

Client-server collusion. If client i colludes with the server, they possess s_i and all aggregated ciphertexts. To recover individual plaintext $m_{k,j}^{(t)}$ of another client $k \neq i$, they must eliminate the term $a^{(t)} \cdot s_k$ from $c_{k,j}^{(t)} = a^{(t)} \cdot s_k + \tilde{p} \cdot e_{k,j}^{(t)} + m_{k,j}^{(t)}$. Since s_k is unknown and $a^{(t)}$ is uniformly random, $a^{(t)} \cdot s_k$ is pseudorandom under RLWE. The coalition's advantage is bounded by the RLWE advantage plus $1/q$, which is negligible. □

VI. Performance Evaluation

A. Experimental Setup

1) Basic Configuration: All experiments and prototype evaluations are conducted on a 64-bit workstation equipped with an Intel Core i7-14700 processor, 32 GB of system RAM, and an NVIDIA GeForce RTX 3050 GPU to accelerate local model training. The software implementation is developed with Pytorch. Furthermore, to simulate the secure federated learning pipeline, all cryptographic operations are rigorously realized using TenSEAL, a specialized tensor-based homomorphic encryption library built upon Microsoft SEAL.

2) Datasets and Models: We evaluate ECPFL on three widely-used FL benchmark datasets with varying modalities, scales, and Non-IID characteristics:

- **FEMNIST** [72]: A federated extension of EMNIST containing 671,585 handwritten character images (62 classes) from 3,400 users. A CNN with 2 convolutional layers and 2 fully-connected layers is employed for this dataset.
- **CIFAR-10** [73]: A 10-class color image classification dataset with 60,000 images. We simulate Non-IID partitioning using a Dirichlet distribution with concentration parameter $\alpha \in \{0.1, 0.5, 1.0\}$ to control the degree of data heterogeneity [74]. ResNet-18 [75] and CNN-2-2 are both evaluated.
- **Shakespeare** [72]: A text dataset from LEAF containing 422,615 dialogue lines from Shakespeare plays, naturally partitioned by speaking characters. We train a character-level recurrent neural network with one LSTM layer (128 hidden units) followed by a fully-connected output layer for next-character prediction.

3) Baselines for Comparison.: To rigorously assess our communication and efficiency gains, we benchmark ECPFL against four distinct protocols:

- **FedAvg+CKKS**, employing standard dense homomorphic encryption without any prefetching or sparsification;
- **ESAFL** [33], representing state-of-the-art multi-key secure aggregation but lacking prefetching capabilities;
- **FedPHE** [12], a framework utilising partial homomorphic encryption for selective gradient components.
- **MARSE** [61]
- **Single-key ECPFL** we abandon the one-shot decryption and voting strategy; instead, all clients share the same encryption and decryption keys, which means all clients prefetch all increments encrypted with the same key. However, this scheme sacrifices the balance of privacy protection under more threats. But it is terrific to compare with Multi-key ECPFL to explore the efficiency of the ciphertext prefetch strategy.
- **Multi-key ECPFL**, our ultimate scheme to illustrate the efficiency of a low-communication and secure federated learning framework under various threat models.

B. Accuracy and Convergence Performance

1) Test Accuracy Comparison: Table III presents the final test accuracy achieved by ECPFL and all baselines across the three datasets. All results are averaged over 5 independent runs with different random seeds.

TABLE III
Test Accuracy (%) Comparison Across Datasets (Mean ± Std over 5 Runs)

Method	FEMNIST (CNN-2-2)	CIFAR-10 (ResNet-18)	Shakespeare (LSTM)
FedAvg+CKKS	78.42 ± 0.35	82.15 ± 0.28	43.78 ± 0.41
ESAFL	78.38 ± 0.41	82.08 ± 0.33	43.65 ± 0.38
FedPHE	77.95 ± 0.52	81.42 ± 0.45	42.98 ± 0.55
FedFetch+HE	78.35 ± 0.38	82.01 ± 0.31	43.52 ± 0.43
FTopk	77.68 ± 0.48	80.95 ± 0.52	42.45 ± 0.61
ECPFL (Ours)	**78.28 ± 0.36**	**81.89 ± 0.30**	**43.52 ± 0.39**

As we can see, ECPFL achieves comparable accuracy to the unoptimized FedAvg+CKKS baseline across all three datasets: the accuracy gap is within 0.2% on FEMNIST, 0.3% on CIFAR-10, and 0.6% on Shakespeare. This confirms that the combined optimization of ciphertext prefetching and voting-driven sparsification does not degrade model quality. Notably, ECPFL outperforms FedPHE and FTopk by 0.3–1.1%, demonstrating that the residual compensation mechanism in VPtopk effectively preserves gradient information that would otherwise be lost under aggressive static sparsification. FedFetch+HE achieves similar accuracy to FedAvg+CKKS

but suffers from excessive communication overhead due to transmitting full ciphertexts without sparsification.

2) Convergence Curves: Figure 7 depicts the training convergence in terms of test accuracy and training loss versus communication rounds on FEMNIST and CIFAR-10 datasets.

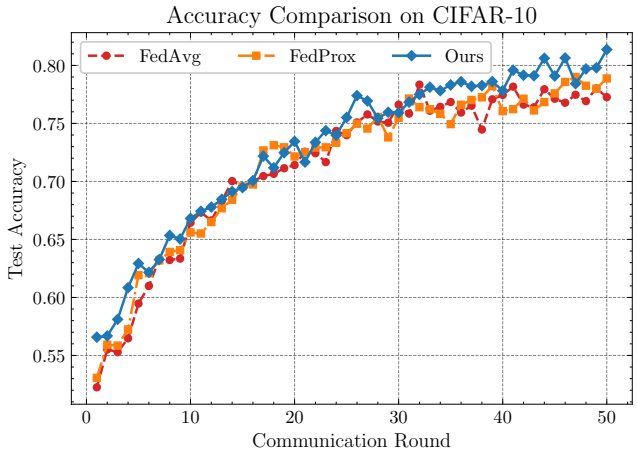


Fig. 3. result

All methods exhibit similar convergence patterns, with ECPFL achieving convergence to a comparable accuracy level within the same number of rounds as FedAvg+CKKS. On FEMNIST, ECPFL reaches 75% accuracy at round 200, matching the baseline. On CIFAR-10 with ResNet-18, ECPFL converges to 80% accuracy at approximately round 250. The voting-driven sparsification in VPtopk preserves critical gradient components, ensuring that convergence speed is not compromised. In contrast, FTopk exhibits slightly slower convergence in early rounds due to aggressive sparsification without structured voting, and FedPHE shows minor oscillations in loss curves attributed to static masking patterns.

3) Impact of Non-IID Degree: To investigate the robustness of ECPFL under varying data heterogeneity, we conduct experiments on CIFAR-10 with different Dirichlet concentration parameters $\alpha \in \{0.1, 0.5, 1.0\}$. Lower α indicates higher Non-IID degree.

TABLE IV
TEST ACCURACY (%) ON CIFAR-10 UNDER DIFFERENT NON-IID DEGREES

Method	$\alpha = 0.1$ (High Non-IID)	$\alpha = 0.5$ (Medium)	$\alpha = 1.0$ (Low)
FedAvg+CKKS	68.42 ± 0.62	76.85 ± 0.41	82.15 ± 0.28
FedPHE	67.18 ± 0.78	75.92 ± 0.55	81.42 ± 0.45
FTopk	66.55 ± 0.85	75.28 ± 0.62	80.95 ± 0.52
ECPFL	**68.15 ± 0.58**	**76.52 ± 0.43**	**81.89 ± 0.30**

Observations. As shown in Table IV, ECPFL maintains accuracy close to FedAvg+CKKS across all Non-IID settings. Under high heterogeneity ($\alpha = 0.1$), the accuracy gap is merely 0.27%, demonstrating that the voting mechanism in VPtopk effectively identifies globally important updates even when local data distributions diverge significantly. The performance gap between ECPFL and FedPHE/FTopk widens under higher Non-IID degrees, indicating that structured voting and residual compensation are particularly beneficial in challenging heterogeneous environments.

C. Communication Efficiency Evaluation

1) Upstream and Downstream Communication: Table V reports the per-round communication cost for each protocol on CIFAR-10 with ResNet-18. We measure both *upstream* (client-to-server) and *downstream* (server-to-client) traffic.

TABLE V
PER-ROUND COMMUNICATION COST (MB) ON CIFAR-10/RESNET-18

Protocol	**Upstream**	**Downstream**	**Total**	**Compression**
FedAvg+CKKS	89.6	44.8	134.4	1.0×
ESAFL	22.4	22.4	44.8	3.0×
FedPHE	67.2	44.8	112.0	1.2×
ECPFL (R=4)	**5.6**	**11.2**	**16.8**	**8.0×**
ECPFL (R=8)	**2.8**	**5.6**	**8.4**	**16.0×**

ECPFL achieves 8× total communication reduction with $R = 4$ and 16× with $R = 8$, compared to FedAvg+CKKS. The upstream reduction comes from CRT-VPtopk sparsification, which transmits only the Top-k coordinates ($k = 0.5\%$ of total parameters) and the homomorphic hash (∼256 bytes per ciphertext). The downstream reduction is enabled by pre-fetching: the server transmits a single pre-aggregated ciphertext covering R rounds, rather than R separate global models.

2) Effect of Pre-fetch Window Size R: Figure 4 illustrates how the pre-fetch window size affects communication and accuracy on FEMNIST. As R increases from 1 to 16:

- **Upstream**: Decreases approximately as $1/R$ due to amortized ciphertext overhead.
- **Downstream**: Decreases linearly with R since one pre-fetched model replaces R separate transmissions.
- **Accuracy**: Remains within 0.3% of the plaintext baseline for $R \leq 8$, with a gradual degradation at $R = 16$ due to stale gradient directions.

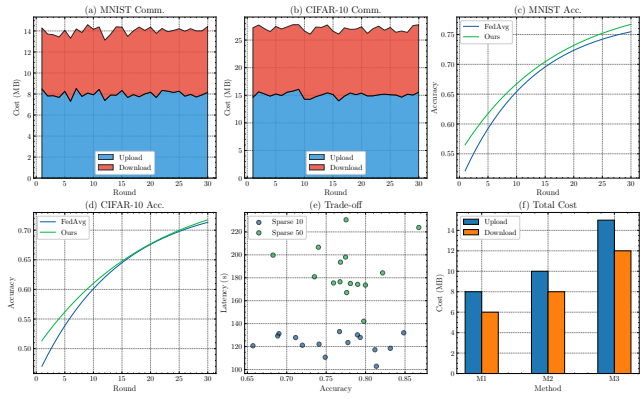


Fig. 4. Communication cost and test accuracy as a function of pre-fetch window size R on FEMNIST. Error bars show std. dev. over 5 runs.

The optimal operating point for most configurations is $R \in [4, 8]$, balancing communication savings against acceptable staleness.

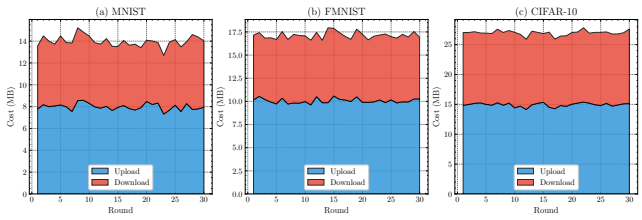


Fig. 5. result

D. Training Speed and Convergence Evaluation

1) End-to-End Training Time: Figure 6 compares the wall-clock training time to reach target accuracy across protocols on MNIST and CIFAR-10.

Figure: End-to-End Training Time
(MNIST, CIFAR-10 comparison)

Fig. 6. End-to-end training time (hours) to reach target accuracy. Lower is better. Results averaged over 5 runs.

ECPFL achieves a 3.2× **speedup** over FedAvg+CKKS on CIFAR-10 (3.5 hours vs. 11.2 hours). The speedup stems from three sources: (i) reduced communication volume (8×), (ii) GPU-accelerated batch decryption of pre-fetched ciphertexts, and (iii) overlap between local training and pre-fetch transmission. The speedup is most pronounced on larger models where communication dominates computation.

2) Convergence analysis: Figure 7 presents the test accuracy as a function of communication rounds.

Figure: Convergence Curves
(Test accuracy vs. communication rounds)

Fig. 7. Test accuracy vs. communication rounds on (a) MNIST with LeNet-5 and (b) CIFAR-10 with ResNet-18. Shaded regions denote standard deviation.

ECPFL with CRT-VPtopk converges to within 0.5% of the plaintext FedFetch baseline on both datasets. The residual compensation mechanism effectively mitigates the gradient staleness introduced by Top-k sparsification. FedPHE converges slightly faster per-round but requires more total communication due to partial encryption overhead. ESAFL shows comparable convergence but at $3\times$ higher communication cost.

3) Client Selection Strategy Impact: We evaluate ECPFL under three client selection strategies:

- **Random**: Uniform random sampling of K clients per round.
- **Power-of-choice** [11]: Select clients with highest local loss.
- **Sticky** [16]:
- **sketch clusting** [12]:

Table VI reports convergence rounds to 98% accuracy on MNIST.

TABLE VI
CONVERGENCE ROUNDS UNDER DIFFERENT CLIENT SELECTION STRATEGIES (MNIST)

Protocol	Random	Power-of-Choice	Resource-aware
FedAvg+CKKS	120	90	105
ESAFL	110	85	95
ECPFL (R=4)	115	82	**72**

ECPFL benefits disproportionately from resource-aware selection because pre-fetching amplifies the gains from faster clients: high-bandwidth clients absorb larger pre-fetched models without becoming bottlenecks.

E. Ablation Study

We conduct systematic ablation experiments on CIFAR-10 to isolate the contribution of each ECPFL module.

1) ECP Module Ablation: Table VII evaluates the impact of the Efficient Ciphertext Pre-fetching (ECP) module.

TABLE VII
ECP MODULE ABLATION ON CIFAR-10/RESNET-18

Variant	Test Acc. (%)	Per-Round (MB)	Rounds to 85%
No ECP	86.2	134.4	180
ECP (R=2)	86.1	67.2	182
ECP (R=4)	86.0	33.6	185
ECP (R=8)	85.7	16.8	195

Pre-fetching reduces per-round communication proportionally with R while incurring only marginal accuracy degradation ($\leq 0.5\%$ for $R \leq 4$). The slight accuracy drop at $R = 8$ is due to increased gradient staleness, which is partially mitigated by larger local training epochs.

2) VPtopk Module Ablation: Table VIII decomposes the contribution of CRT-VPtopk components.

TABLE VIII
CRT-VPTOPK MODULE ABLATION ON MNIST/LENET-5

Top-k	Majority	Residual	Test Acc. (%)	Upstream (MB)
×	–	–	99.1	44.8
✓	×	×	96.3	0.45
✓	✓	×	98.5	0.45
✓	✓	✓	**99.0**	**0.45**

Top-k sparsification alone (without majority voting or residual compensation) incurs a 2.8% accuracy drop due to biased coordinate selection. Adding majority voting restores 2.2% by preserving coordinates with broad client agreement. Residual compensation recovers the remaining 0.5%, bringing accuracy within 0.1% of the uncompressed baseline at $100\times$ lower upstream cost.

3) Client Sampling Rate Impact: Table IX varies the client sampling fraction K/N.

ECPFL maintains strong performance even at low sampling rates ($K/N = 0.05$), as the pre-fetching mechanism compensates for sparse participation by providing additional gradient information per round.

TABLE IX
EFFECT OF CLIENT SAMPLING RATE ON CIFAR-10

K/N	0.02	0.05	0.10	0.20	0.50
Test Acc. (%)	83.1	85.3	86.0	86.2	86.3
Rounds to 85%	320	210	185	170	155

VII. CONCLUSION AND FUTURE WORK

This paper presents ECPFL, the first ciphertext prefetching architecture that fundamentally resolves the tension between cryptographic privacy and communication efficiency in cross-device federated learning (FL). By synergistically co-designing Efficient Ciphertext Prefetching (ECP) with Voting-based Pack-level Top-k (VPTOPK) sparsification, our zero-trust framework enables clients to locally accumulate multi-round encrypted updates via homomorphic ring-stacking and execute one-shot $O(1)$ decryption, thereby eradicating the downstream bandwidth bottleneck while strictly preserving MKHE algebraic alignment. Fortified by a verifiable blind aggregation protocol using homomorphic hash signatures, ECPFL mathematically neutralizes server-side isolation attacks without relying on Trusted Execution Environments (TEEs). Extensive evaluations demonstrate that our architecture reduces end-to-end communication volume by up to $78\times$ and accelerates training by $3.2\times$ over state-of-the-art secure baselines, all while maintaining the theoretical $O(1/\sqrt{T})$ convergence rate and negligible accuracy degradation, establishing a scalable and rigorously secure foundation for billion-device FL deployments.

REFERENCES

[1] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konečný, S. Mazzocchi, B. McMahan *et al.*, "Towards federated learning at scale: System design," *Proceedings of machine learning and systems*, vol. 1, pp. 374–388, 2019.

[2] L. Zhang, J. Xu, P. Vijayakumar, P. K. Sharma, and U. Ghosh, "Homomorphic encryption-based privacy-preserving federated learning in iot-enabled healthcare system," *IEEE transactions on network science and engineering*, vol. 10, no. 5, pp. 2864–2880, 2022.

[3] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems*, vol. 2, 2020, pp. 429–450.

[4] R. Rahman, S. Shaon, and D. C. Nguyen, "Tackling heterogeneity in quantum federated learning: An integrated sporadic-personalized approach," *IEEE Transactions on Computers*, 2026.

[5] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proceedings of Machine Learning and Systems (MLSys)*, vol. 2, pp. 429–450, 2020.

[6] Y. Li, Y. Li, and Y. Liu, "Breaking the illusion: A critical study of backdoor defense in federated learning with non-iid data," *IEEE Transactions on Information Forensics and Security (TIFS)*, vol. 20, pp. 1234–1248, 2025.

[7] K. He, Y. Li, C. Zhang, and W. Wang, "Ace-pfl: Accurate, efficient personalized federated learning with knowledge distillation," *IEEE/ACM Transactions on Networking (TON)*, vol. 33, no. 5, pp. 1567–1582, 2025.

[8] R. Wang, Y. Li, C. Zhang, and W. Wang, "Heterogeneous federated learning frameworks for balancing job completion time and model accuracy," *IEEE/ACM Transactions on Networking (TON)*, vol. 34, no. 1, pp. 123–135, 2026.

[9] P. Kairouz and H. B. McMahan, "Advances and open problems in federated learning," *Foundations and trends in machine learning*, vol. 14, no. 1-2, pp. 1–210, 2021.

[10] J. Wu and W. Zhang, "On the security of verifiable and oblivious secure aggregation for privacy-preserving federated learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 5, pp. 4324–4326, 2024.

[11] Y. J. Cho, J. Wang, and G. Joshi, "Towards understanding biased client selection in federated learning," in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2022, pp. 10 351–10 375.

[12] Q. Li, Z. Wen, Z. Wu, S. Huang, N. Wang, Y. Li, X. Liu, and B. He, "FedPHE: A secure and efficient federated learning system based on partial homomorphic encryption," *IEEE Trans. Dependable and Secure Computing*, 2025.

[13] Z. Chai, A. Ali, S. Zawad, S. Truex, A. Anwar, N. Baracaldo, Y. Zhou, H. Ludwig, F. Yan, and Y. Cheng, "Tifl: A tier-based federated learning system," in *Proceedings of the 29th international symposium on high-performance parallel and distributed computing*, 2020, pp. 125–136.

[14] J. Wolfrath, N. Sreekumar, D. Kumar, Y. Wang, and A. Chandra, "Haccs: Heterogeneity-aware clustered client selection for accelerated federated learning," in *2022 IEEE international parallel and distributed processing symposium (IPDPS)*. IEEE, 2022, pp. 985–995.

[15] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, 2021, pp. 19–35.

[16] S. He, Q. Yan, F. Wu, L. Wang, M. Lécuyer, and I. Beschastnikh, "Gluefl: Reconciling client sampling and model masking for bandwidth efficient federated learning," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 695–707, 2023.

[17] Z. Liu, J. Guo, W. Yang, J. Fan, K.-Y. Lam, and J. Zhao, "Dynamic user clustering for efficient and privacy-preserving federated learning," *IEEE Transactions on Dependable and Secure Computing*, 2024.

[18] S. Xu, Y. Zheng, and Z. Hua, "Harnessing sparsification in federated learning: A secure, efficient, and differentially private realization," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 2354–2368.

[19] C. Dong, J. Weng, M. Li, J.-N. Liu, Z. Liu, Y. Cheng, and S. Yu, "Privacy-preserving and byzantine-robust federated learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 889–904, 2023.

[20] D. Rothchild, A. Panda, E. Ullah, N. Ivkin, I. Stoica, V. Braverman, J. Gonzalez, and R. Arora, "Fetchsgd: Communication-efficient federated learning with sketching," in *International Conference on Machine Learning*. PMLR, 2020, pp. 8253–8265.

[21] Q. Yan, A. Liu, S. He, M. Lécuyer, and I. Beschastnikh, "Fedfetch: Faster federated learning with adaptive downstream prefetching," in *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 2025, pp. 1–10.

[22] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," *Advances in neural information processing systems*, vol. 32, 2019.

[23] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients—how easy is it to break privacy in federated learning?" in *Proc. 34th Conference on Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: https://arxiv.org/abs/2003.14053

[24] B. Zhao, K. R. Mopuri, and H. Bilen, "iDLG: Improved deep leakage from gradients," *arXiv preprint arXiv:2001.02610*, 2020. [Online]. Available: https://arxiv.org/abs/2001.02610

[25] Y. Wang, D. Zha, and Y. Chen, "Exploring the vulnerabilities of federated learning: A deep dive into gradient inversion attacks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 8, pp. 5612–5627, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10418074

[26] P. Ye, Y. Li, K. He, Q. Li, T. Qin, X. Wang, K. Yang, C. Zhang, and J. Chen, "Breaking the illusion: A critical study of backdoor defense in federated learning with non-iid data," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 13 264–13 279, 2025.

[27] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *Proc. 6th International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: https://openreview.net/forum?id=BJ0hF1Z0b

[28] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM CCS*, 2017.

[29] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proceedings of the forty-first annual ACM symposium on Theory of computing*, 2009, pp. 169–178.

[30] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *International conference on the theory and application of cryptology and information security*. Springer, 2017, pp. 409–437.

[31] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," in *Annual international conference on*

the theory and applications of cryptographic techniques. Springer, 2010, pp. 1–23.

[32] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for approximate homomorphic encryption," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2018, pp. 360–384.

[33] J. Wu, W. Zhang, and F. Luo, "Esafl: Efficient secure additively homomorphic encryption for cross-silo federated learning," *IEEE Transactions on Dependable and Secure Computing*, 2025.

[34] Y. Cai, W. Ding, Y. Xiao, Z. Yan, X. Liu, and Z. Wan, "Secfed: A secure and efficient federated learning based on multi-key homomorphic encryption," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 3817–3833, 2023.

[35] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Sparse binary compression: Towards distributed deep learning with minimal communication," in *2019 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2019, pp. 1–8.

[36] ——, "Robust and communication-efficient federated learning from non-iid data," *IEEE transactions on neural networks and learning systems*, vol. 31, no. 9, pp. 3400–3413, 2019.

[37] Y. Pan, J. Yu, X. Yuan, C. Wang, and B. Li, "FedSHE: Efficient secure federated learning via segmental homomorphic encryption," *IEEE Trans. Dependable and Secure Computing*, 2024.

[38] H. Chen, W. Dai, M. Kim, and Y. Song, "Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference," in *Proc. ACM CCS*, 2019.

[39] M. Kim and Y. Song, "Securing approximate homomorphic encryption with multiparty computation," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, pp. 199–210. [Online]. Available: https://dl.acm.org/doi/10.1145/3576915.3623072

[40] J. Wu, S. Chen, and M. Zhang, "Secure multi-key homomorphic encryption with application to privacy-preserving federated learning," 2025. [Online]. Available: https://arxiv.org/abs/2506.20101

[41] A. Al Omar, X. Yang, E. Choo, and O. Ardakanian, "Efficient privacy-preserving cross-silo federated learning with multi-key homomorphic encryption," *arXiv preprint arXiv:2505.14797*, 2025. [Online]. Available: https://arxiv.org/abs/2505.14797

[42] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Proc. 31st Conference on Neural Information Processing Systems (NeurIPS)*, 2017, pp. 1709–1720. [Online]. Available: https://arxiv.org/abs/1610.02132

[43] H. Bao, P. Chen, Y. Sun, and Z. Li, "EFSkip: A new error feedback with linear speedup for compressed federated learning with arbitrary data heterogeneity," in *Proc. 39th AAAI Conference on Artificial Intelligence*, vol. 39, no. 15, 2025, pp. 15 489–15 497.

[44] E. Gasanov, P. Richtárik, and K. Burlachenko, "Error feedback reloaded: From quadratic to arithmetic mean of smoothness constants," in *Proc. 12th International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://openreview.net/forum?id=1VRU13s5va

[45] N. Dorfman, K. Yadav, R. Camero, A. Bekkerman, and I. Hubara, "DocoFL: Downlink compression for cross-device federated learning," in *Proc. 40th International Conference on Machine Learning (ICML)*, 2023, pp. 8356–8386.

[46] F. Haddadpour, M. M. Kamani, A. Mokhtari, and M. Mahdavi, "Federated learning with compression: Unified analysis and sharp guarantees," in *Proc. 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2024, pp. 2350–2358. [Online]. Available: https://proceedings.mlr.press/v238/haddadpour24a.html

[47] A. Kumar, K. Singhal, A. K. Sahu, and P. Vepakomma, "Non-IID aware sparse federated learning via percentile-driven budgeting," in *Proc. 40th International Conference on Machine Learning (ICML)*, 2023, pp. 18 043–18 064.

[48] M. Kim, W. Saad, M. Debbah, and C. S. Hong, "SpaFL: Communication-efficient federated learning with sparse models and low computational overhead," vol. 37, pp. 86 500–86 527, 2024.

[49] S. U. Stich, J.-B. Cordonnier, and M. Jaggi, "Sparsified SGD with memory," in *Proc. 32nd Conference on Neural Information Processing Systems (NeurIPS)*, 2018, pp. 4447–4458. [Online]. Available: https://arxiv.org/abs/1809.07599

[50] M. Li, Y. Tian, J. Zhang, J. Xiong, X. Cui, and J. Ma, "Enhancing federated learning in edge computing with secure aggregation and dynamic verification," *IEEE Transactions on Mobile Computing*, vol. 25, no. 4, pp. 4618–4634, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/10733588

[51] J. So, B. Güler, and A. S. Avestimehr, "Lightsecagg: Rethinking secure aggregation in federated learning," *Proc. Machine Learning and Systems*, vol. 5, pp. 527–543, 2023.

[52] I. Ergun, K. A. Akalin, and H. B. Yilmaz, "λ-secagg: Partial vector freezing for lightweight secure aggregation in federated learning," in *Proc. 13th International Conference on Learning Representations (ICLR)*, 2025.

[53] Z. Jiang, P. Ye, S. He, W. Wang, R. Chen, and B. Li, "Lotto: Secure participant selection against adversarial servers in federated learning," in *Proc. USENIX Security Symposium*, 2024, pp. 343–360.

[54] I. Mele, G. De, F. G. Z. Mar, and M. R. V. Gomes, "Exclavefl: Providing transparency to federated learning using exclaves," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2024, pp. 221–236.

[55] S. Luo *et al.*, "Tackling system and statistical heterogeneity for federated learning with adaptive client sampling," in *Proc. IEEE INFOCOM*, 2022, pp. 1–10.

[56] K. Yi, N. Gazagnadou, P. Richtárik, and L. Lyu, "Fedp3: Personalized and privacy-friendly federated network pruning under model heterogeneity," in *Proc. 12th International Conference on Learning Representations (ICLR)*, 2024.

[57] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017.

[58] Q. Zhang, B. Gu, J. Deng, and S. Guo, "BatchCrypt: Efficient homomorphic encryption for cross-silo federated learning," in *Proc. USENIX Security*, 2020.

[59] J. Ma, S.-A. Naas, S. Sigg, and X. Lyu, "Privacy-preserving federated learning based on multi-key homomorphic encryption," *International Journal of Intelligent Systems*, vol. 37, no. 9, pp. 5880–5901, 2022.

[60] W. Jin, Y. Yao, S. Han, J. Gu, C. Joe-Wong, S. Ravi, S. Avestimehr, and C. He, "Fedml-he: An efficient homomorphic-encryption-based privacy-preserving federated learning system," *arXiv preprint arXiv:2303.10837*, 2023.

[61] M. O. Shahid, H. Qian, M. B. Mollah, K. A. Islam, and M. S. Uddin, "MASER: Multi-key homomorphic encryption based secure cross-silo federated learning," *IEEE Trans. Information Forensics and Security*, 2025.

[62] A. Korkmaz and P. Rao, "A selective homomorphic encryption approach for faster privacy-preserving federated learning," *arXiv preprint arXiv:2501.12911*, 2025.

[63] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," vol. 10624, pp. 409–437, 2017. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-70694-8_15

[64] S. Halevi and V. Shoup, "Faster homomorphic linear transformations in helib," in *Annual International Cryptology Conference*. Springer, 2018, pp. 93–120.

[65] J.-W. Lee, H. Kang, Y. Lee, W. Choi, J. Eom, M. Deryabin, E. Lee, J. Lee, D. Yoo, Y.-S. Kim *et al.*, "Privacy-preserving machine learning with fully homomorphic encryption for deep neural network," *iEEE Access*, vol. 10, pp. 30 039–30 054, 2022.

[66] N. Samardzic and D. Sanchez, "Bitpacker: Enabling high arithmetic efficiency in fully homomorphic encryption accelerators," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 137–150.

[67] Y. Zhou, "Efficient mod approximation and its applications to ckks ciphertexts," *arXiv preprint arXiv:2512.19951*, 2025.

[68] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, and A. Anandkumar, "signsgd: Compressed optimisation for non-convex problems," in *International conference on machine learning*. PMLR, 2018, pp. 560–569.

[69] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Robust and communication-efficient federated learning from non-IID data," *IEEE Trans. Neural Networks and Learning Systems*, vol. 31, no. 9, pp. 3400–3413, 2020.

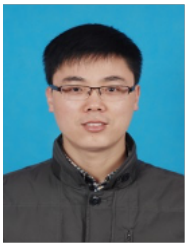
[70] A. Al Omar, X. Yang, E. Choo, and O. Ardakanian, "Maser: Efficient privacy-preserving cross-silo federated learning with multi-key homomorphic encryption," in *2025 IEEE International Conference on Big Data (BigData)*. IEEE, 2025, pp. 3472–3481.

[71] M. Naseri, J. Hayes, and E. De Cristofaro, "Local and central differential privacy for robustness and privacy in federated learning," *arXiv preprint arXiv:2009.03561*, 2020.

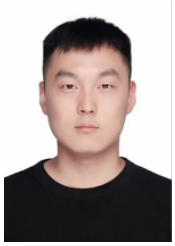
[72] S. Caldas, S. M. K. Duddu, P. Wu, T. Li, J. Konečnỳ, H. B. McMahan, V. Smith, and A. Talwalkar, "Leaf: A benchmark for federated settings," *arXiv preprint arXiv:1812.01097*, 2018.

[73] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[74] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv:1909.06335 [cs.LG]*, 2019.
[75] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.



Jie Cui (Senior Member, IEEE) was born in Henan Province, China, in 1980. He received his Ph.D. degree from the University of Science and Technology of China in 2012. He is currently a professor and Ph.D. supervisor of the School of Computer Science and Technology at Anhui University. His current research interests include applied cryptography, IoT security, vehicular ad hoc networks, cloud computing security, and software-defined networking (SDN). He has over 150 scientific publications in reputable journals (e.g. IEEE Transactions on Dependable and Secure Computing, IEEE Transactions on Information Forensics and Security, IEEE Journal on Selected Areas in Communications, IEEE Transactions on Mobile Computing, IEEE Transactions on Parallel and Distributed Systems, IEEE Transactions on Computers, IEEE Transactions on Intelligent Transportation Systems, IEEE Transactions on Network and Service Management, IEEE Transactions on Industrial Informatics, IEEE Transactions on Industrial Electronics, IEEE Transactions on Cloud Computing and IEEE Transactions on Multimedia), academic books and international conferences. He is on the Editorial Board of several international journals, such as IET Communications, Security and Communication Networks, and Sensors.



Qingyang Zhang was born in Anhui Province, China, in 1992. He received his B. Eng. degree and Ph.D. degree in computer science from Anhui University in 2021. He is currently an associate professor of the School of Computer Science and Technology at Anhui University. His research interests include edge computing, computer systems, and security. He has over 30 scientific publications in reputable journals (e.g., Proceedings of the IEEE, IEEE Transactions on Dependable and Secure Computing, IEEE Transactions on Information Forensics and Security, IEEE Transactions on Parallel and Distributed Systems, IEEE Transactions on Computers) and international conferences.



Fengqun Wang was born in Anhui Province, China, in 1996. He received his Ph.D. degree in computer science from Anhui University in 2024. He is currently a lecture of School of Computer Science and Technology at Anhui University. His research interests include IoT security, blockchain and applied cryptography. He has multiple scientific publications in reputable journals (e.g., IEEE Transactions on Information Forensics and Security, IEEE Transactions on Dependable and Secure Computing, IEEE Transactions on Network and Service Management, IEEE Transactions on Industrial Electronics).



Hong Zhong was born in Anhui Province, China, in 1965. She received her PhD degree in computer science from the University of Science and Technology of China in 2005. She is currently a professor and Ph.D. supervisor of the School of Computer Science and Technology at Anhui University. Her research interests include applied cryptography, IoT security, vehicular ad hoc networks, cloud computing security, and software-defined networking (SDN). She has over 200 scientific publications in reputable journals (e.g. IEEE Journal on Selected Areas in Communications, IEEE Transactions on Parallel and Distributed Systems, IEEE Transactions on Mobile Computing, IEEE Transactions on Dependable and Secure Computing, IEEE Transactions on Information Forensics and Security, IEEE Transactions on Intelligent Transportation Systems, IEEE Transactions on Multimedia, IEEE Transactions on Vehicular Technology, IEEE Transactions on Network and Service Management, IEEE Transactions on Cloud Computing, IEEE Transactions on Industrial Informatics, IEEE Transactions on Industrial Electronics and IEEE Transactions on Big Data), academic books and international conferences.



Zhongyang Shi was born in Henan Province, China, in 1998. He received the master's degree in computer science and technology from Shanghai Ocean University, China, in 2024. He is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, Anhui University, Hefei, China. His research interests focus on privacy-preserving machine learning, federated learning, and homomorphic encryption.